バックトレース法プログラムの改善点

backtrace.f90 から backtrace_rev.f90 への変更のまとめ

Claude Code による整理（豊田政弘氏との検討記録）

2026 年 10 月 19 日

要約

- `backtrace_rev.f90` は，旧版 `backtrace.f90` と同じ手法・同じ判定式のまま，虚音源の拾い漏れ，メモリを壊す危険，計算の遅さを直したものである．
- 主な改善は 6 つ：(1) 音線追跡を 1 回にした，(2) 反射履歴に平面の番号を記録する，(3) 音線数を計算したい時間長から決める，(4) 重複経路の削除を整列方式にした，(5) 出力の順序を固定した，(6) 音線方向の式の 1 ずれを直した．
- 音線 100 万本の比較で，計算時間は 50 秒から 2.4 秒になり，見つかった可視な虚音源は 21,490 個から 70,554 個に増えた．増えた分を含む全経路が実在することを，プログラムとは独立な検算で確認した．

1　対象と手法の概要

1.1　バックトレース法

対象の手法は Vorländer[1] の音線法・虚像法の組み合わせである．虚像法（壁で鏡に映した「虚音源」を並べて反射音を求める方法）は到達時間を正確に求められるが，反射次数とともに候補の虚音源が指数的に増え，そのほとんどは受音点から見えない．そこで音線法で「実際に受音点に届く虚音源」だけを選び出す．手順は次の 4 段階である．

1. 音源から全方向に音線を飛ばし，半径 r の受音球を通過した音線について，それまでに反射した壁の順番（反射履歴）を記録する．
2. 同じ反射履歴の重複を除く．
3. 反射履歴に沿って音源を順に鏡に映し，虚音源の位置を求める．
4. 受音球の中心から虚音源へ逆にたどり，記録どおりの壁で反射して音源に着くかを確かめる（受音球に大きさがあるために紛れ込む，中心からは見えない虚音源を除く）．

各経路について，反射回数，到達時間，虚音源の方向，6 帯域の反射エネルギーを出力する．

1.2 入力

形状は `hall1.dxf`（38 枚の三角形, 11 枚の平面, 体積 19,000 m^3, 表面積 4,925 m^2, 平均自由行程 15.4 m），材料は `absorption.csv`（垂直入射吸音率，6 帯域）．音源 $(5, 10, 4.2)$ m，受音点 $(20, 10, 1.2)$ m，受音球の半径 1 m，音速 340 m/s．法線を反転したあとの全面の法線が室内を向き，音源・受音点が室内にあることを確認した．

2 変更していない部分

次の部分は，結果が旧版と一致するように意図して残した．

- 吸音率 csv と DXF 形状の読み込み，法線の計算（旧版 75〜263 行目を行単位で流用）．
- 音線が三角形の内側に当たるかの判定式，受音球の判定式．
- 斜入射の反射率の式．垂直入射吸音率 α_0 から，壁を実数インピーダンス ζ の局所作用面とみなして

$$r_0 = \sqrt{1-\alpha_0}, \qquad \zeta = \frac{1+r_0}{1-r_0}, \qquad R(\theta) = \left| \frac{\zeta\cos\theta - 1}{\zeta\cos\theta + 1} \right|^2 \tag{1}$$

  とする．csv の値は垂直入射吸音率であることを確認済みで，この扱いでよい．
- 出力ファイルの形式（11 列：反射回数，到達時間，虚音源の方向 xyz，6 帯域のエネルギー）．

出力のエネルギーは壁での反射損失の積のみで，距離減衰 $1/r^2$，空気吸収，音源の指向性は含まない（後処理で加える）．

3 改善点

3.1 改善 1：音線追跡を 1 回にした

■旧版　受音した反射履歴を入れる配列の大きさを前もって決めるため，同じ追跡を 2 回行っていた．1 回目（並列）で受音の件数だけを数えて配列を確保し，2 回目（1 スレッド）で同じ追跡をやり直して履歴を書き込む．

■問題　1 回目と 2 回目はコンパイラが別々に最適化した別の機械語で動く．浮動小数点の端数がわずかに違って受音球の縁をかすめる音線の判定が変わり，2 回目の件数のほうが多くなると，確保した配列の外に書き込む．エラーは出ず，メモリが壊れたまま計算が続く．また 2 回目は 1 スレッドなので，多コアの計算機でも 1 コアしか使わない．

■修正版　追跡は 1 回（並列）だけである．各スレッドが自分専用の作業領域（`histbuf` 型）に履歴を積み，一杯になったら 2 倍に広げる（`append_hist`）．最後に全スレッドの分を 1 つにまとめる．件数を前もって知る必要がないので，食い違いは起こらない．

3.2　改善 2：反射履歴に平面の番号を記録する

■旧版　反射履歴には当たった三角形の番号を記録し，逆追跡で当たった三角形が記録と同じ番号でなければ経路を捨てていた．

■問題　虚音源の位置はどの平面で鏡に映したかだけで決まり，平面のどの三角形に当たったかには関係しない．受音球の端を通った音線が床の三角形 B に当たって記録されても，受音点の中心からの経路は同じ床の三角形 A に当たることがある．このとき経路は正しいのに，番号が違うので捨てられる．高次の経路は反射を重ねるので，どこか 1 回でも食い違えば捨てられ，拾い漏れは次数とともに急に増える．

■修正版

- 計算の最初に，法線と位置が一致する三角形を同じ平面にまとめて番号を振る．DXF に 0.001 mm 程度の座標のずれがあるため，許容値 $1 - \boldsymbol{n}_1 \cdot \boldsymbol{n}_2 \le 10^{-10}$，$|d_1 - d_2| \le 10^{-4}\,\mathrm{m}$ （d は平面の方程式 $ax + by + cz + d = 0$ の定数項）を設けた．このホールでは 38 枚が 11 枚の平面にまとまる．
- 反射履歴には平面の番号を記録する．
- 逆追跡では，当たった三角形が記録と同じ平面上にあれば有効とする．
- 反射率は，逆追跡で実際に当たった三角形の材料で計算する．

3.3　改善 3：音線数を計算したい時間長から決める

■旧版　音線数 N（1000 万本）と反射回数（100 回）を直接指定していた．

■問題　距離 ct の虚音源から半径 r の受音球に届く音線数の期待値は，文献 [1] の式 (8) より

$$k = \frac{N r^2}{4(ct)^2} \tag{2}$$

である．1 本も届かなければその虚音源は見つからず，その確率は e^{-k} である．旧版の設定では反射 100 回付近（約 4.5 秒）で $k \approx 1$，見落とす確率は約 37 % だった．

■修正版

- パラメータを，計算する時間長 `tmax`（既定 3 s）と，時刻 `tmax` での期待値 `nhitmin`（既定 3 本）に変えた．音線数は文献 [1] の式 (9) を一般化した

  $$N = \frac{4\,k_{\min}\,(c\,t_{\max})^2}{r^2} \tag{3}$$

  で自動的に決まる．既定値では $N \approx 1250$ 万本，時刻 `tmax` での見落とし確率は $e^{-3} \approx 5\,\%$ で，それより前の時刻ではさらに小さい．
- 伝搬距離が $c\,t_{\max}$ を超えた音線は追跡を打ち切り，出力も到達時間 `tmax` 以内に限る．
- `nref` は反射回数の上限（安全装置）とし，`tmax` 以内なのに上限に達した音線の数を表示する．
- 計算の最初に，音線数と，期待値が 1 本になる時刻を表示する．

3.4 改善 4：重複経路の削除を整列方式にした

■旧版　反射回数ごとにグループに分け，グループ内の全ペアを比べていた．

■問題　計算量が件数の 2 乗に比例する．件数が 10 倍になると時間は 100 倍である．

■修正版　反射履歴を辞書式に整列する（1 回目の反射の平面番号で並べ，同じなら 2 回目で並べる，という順．ボトムアップのマージソート）．整列すると同じ履歴は必ず隣どうしに並ぶので，隣と比べるだけで重複を除ける．計算量は $n \log n$ 程度に下がる．あわせて履歴の配列を（反射順，経路番号）の向きにし，1 つの経路の履歴がメモリ上で連続するようにした．

3.5 改善 5：出力の順序を固定した

旧版は並列の逆追跡の中で各スレッドがその場でファイルに書き込んでいたため，同じ入力でも実行のたびに行の順序が変わった．修正版は結果をいったん配列に貯め，計算後に 1 スレッドで反射履歴の辞書式順に書き出す．同じ入力なら同じ出力になる．

3.6 改善 6：音線方向の式の 1 ずれを直した

音線方向は球面上のフィボナッチ格子（らせん状に点を並べて球面をほぼ均等に覆う方法）で決めている．i 番目（$i = 1, \dots, N$）の音線の z 成分は

$$\text{旧版：} z_i = \frac{2i+1}{N} - 1, \qquad \text{修正版：} z_i = \frac{2i-1}{N} - 1 \tag{4}$$

である．旧版では $i = N$ で $z = 1 + 1/N > 1$ となり，$\sqrt{1 - z^2}$ が NaN になってその音線が失われていた．

3.7 そのほかの整理

- 最も手前の壁を探す約 70 行の処理は，旧版では 3 か所にコピーされていた．サブルーチン `nearest_wall` にまとめ，3 か所の判定が必ず同じになるようにした（計算内容は同じ）．
- 全音線の方向を保存する配列（1000 万本で 240 MB）をやめ，使うたびにその場で計算する．
- 壁に当たらず室外へ抜けた音線の数を表示する．形状データに穴がないかの点検に使える．
- 虚音源を鏡に映す式で，平面からの距離の絶対値ではなく符号付きの距離を使う．正しい経路では虚音源は必ず平面の表側にあるので，結果は同じである．

3.8 パラメータの対応

旧版	修正版	意味
`nray = 10000000`	`tmax = 3.0`, `nhitmin = 3.0`	音線数は式 (3) で自動計算
`nref = 100`	`nref = 100`	反射回数の上限（安全装置）
—	`tolnm = 1e-10`, `told = 1e-4`	同一平面とみなす許容値

その他のパラメータ（音源・受音点の位置，受音球の半径，音速，入出力ファイル名，DXF の設定）は旧版と同じである．

4　検証

4.1　条件

両版をそれぞれ ifort 18.0.5（`/O3 /QxHost /Qopenmp`，32 スレッド）でビルドし，音線 100 万本，受音球の半径 1 m，反射回数の上限 100 回で比べた（修正版は `tmax` $= 4.5$ s，`nhitmin` $= 0.1068$ として音線数を約 100 万本にそろえた）．

4.2　結果

	旧版	修正版
計算時間	50 s	2.4 s
可視な虚音源（到達時間 4.5 s 以内）	21,058	70,554
虚音源の位置の重複	0	0
独立な検算で実在しない経路	0	0

- **低次**：反射 5 回までの虚音源は，両版で数も位置も完全に一致した．共通する経路のエネルギーの差は相対 4×10^{-11} 以下である．
- **独立な検算**：出力された全経路について，受音点から虚音源の方向へ音線を出し，Python で別に実装した交差判定で壁ごとに鏡面反射させ，記録どおりの反射回数と距離で音源に着くかを調べた．両版の全経路が実在する経路だった．すなわち修正版で増えた経路はすべて本物である．
- **高次**：図 1 のとおり，旧版は届く音線が十分ある時間帯（0.5〜1 s，期待値 2〜9 本）でも約 25 %，1〜1.5 s では約 60 % の経路を捨てていた．改善 2 の三角形番号の食い違いによる拾い漏れである．
- 旧版だけにある 577 経路は，すべて期待値が小さい後半（主に 1 s 以降）にあり，改善 3 で述べた確率的な見落としである．音線の配り方がわずかに違うため，どちらの版にも生じる．

4.3　既定の設定での実行

`tmax` $= 3$ s，`nhitmin` $= 3$ では，音線 12,484,800 本，計算時間 15 s，可視な虚音源 187,207 個だった．反射回数の上限に達した音線は 968 本，壁の継ぎ目をすり抜けて室外へ抜けた音線は 13 本で，影響は無視できる．

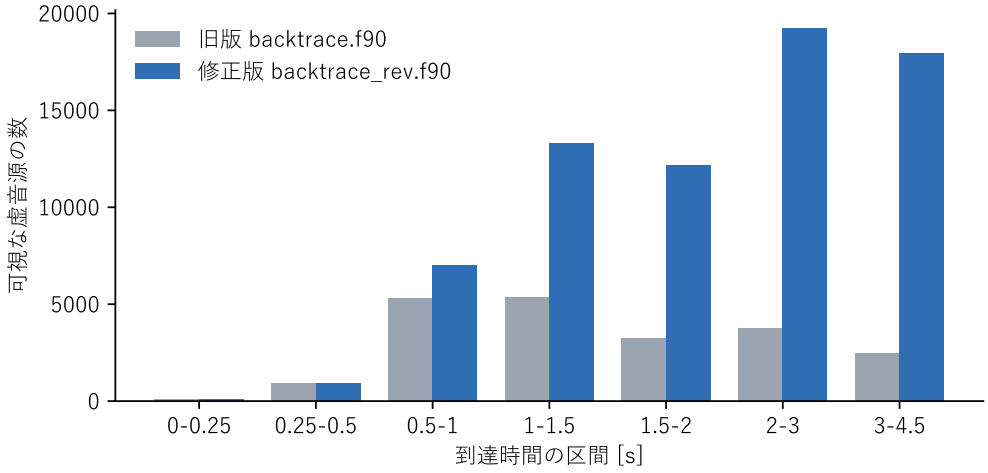


図 1　到達時間の区間ごとの可視な虚音源の数（音線 100 万本）

5　利用上の注意

- 応答をより長く計算するときは `tmax` を変える．音線数は時間長の 2 乗に比例する（5 s なら約 3500 万本）．上限に達した音線数の表示が多ければ `nref` を増やす．
- `told`（0.1 mm）はこのホールの DXF に合わせた値である．別の形状で，本来は別の平面なのに 0.1 mm 以内に重なる面がある場合は見直す．
- 出力のエネルギーには距離減衰，空気吸収，音源の指向性が含まれないので，後処理で加える．
- ビルドは `BACKTRACE/build_ifort.bat`（旧版と修正版の両方を作る）．

参考文献

[1] M. Vorländer, “Simulation of the transient and steady-state sound propagation in rooms using a new combined ray-tracing/image-source algorithm,” J. Acoust. Soc. Am. **86**(1), 172–178 (1989).